

Product Requirements Document (PRD)

Product Name

GoPlan

Product Tagline

Plan anything. Track everything.

1. Purpose & Vision

GoPlan is a flexible, modular, and scalable **planning and execution platform** designed to support **any type of initiative** — software projects, events, collections, operations, marketing campaigns, construction work, AI/ML deployments, and community initiatives.

The vision of GoPlan is to become a **universal planning system** that adapts to how people actually work, instead of forcing teams into rigid methodologies.

2. Problem Statement

Most project management tools today are: - Too opinionated (software-only, Agile-only, Scrum-first) - Too rigid (fixed fields, fixed workflows) - Fragmented (different tools for different project types)

As a result, teams struggle with: - Adapting tools to non-standard workflows - Onboarding non-technical users - Managing multiple project types in one organization

Users want: - One platform for all planning needs - Flexible structure without added complexity - Multiple views (Kanban, Gantt, List, Calendar) - Fast setup using reusable templates

3. Goals & Objectives

Business Goals

- Position GoPlan as a universal planning platform
- Serve SMEs, enterprises, and community organizations
- Enable future monetization via templates, automation, and AI capabilities

User Goals

- Plan work clearly and confidently
 - Collaborate across roles and teams
 - Customize workflows without technical setup
 - Gain visibility into progress, risks, and timelines
-

4. Target Users & Personas

1. Planner (Project Owner)

- Creates and owns plans or projects
- Defines structure, milestones, and priorities

2. Contributor (Team Member)

- Executes assigned tasks
- Updates progress and status

3. Observer (Stakeholder / Viewer)

- Monitors progress and outcomes
- Read-only access

4. Workspace Admin (Optional)

- Manages users, permissions, and templates
-

5. Core Concepts & High-Level Data Model

Key Entities

- Workspace
- Plan (Project)
- Phase / Module (optional)
- Task
- Sub-task
- Milestone
- Dependency
- Custom Field
- Comment & Activity Log

Design principle: **Everything is optional except the Plan and Task.**

6. Key Features (MVP)

6.1 Plan Management

- Create, edit, archive plans
- Plan owner, description, timeline
- Plan status: Draft, Active, On Hold, Completed

6.2 Flexible Structure

- Optional phases or modules
- Unlimited tasks and sub-tasks
- Task dependencies (optional by default)

6.3 Task Management

- Assign contributors
- Due dates and priorities
- Customizable status workflow
- Tags and labels

6.4 Multiple Views

- Kanban Board (workflow-driven)
- List View (structured)
- Calendar View (time-based)
- Gantt View (milestones & dependencies)

6.5 Customization

- Custom fields (text, number, date, dropdown)
- Custom statuses per plan
- Reusable plan templates

6.6 Collaboration

- Task-level comments
- @Mentions
- Activity timeline per plan

6.7 Notifications

- Assignment alerts
 - Due date reminders
 - Status and milestone changes
-

7. Non-Functional Requirements

- Highly scalable (large plans & teams)
 - Secure (RBAC, audit logs)
 - Responsive UI (<200ms interactions)
 - Mobile-first and cross-platform ready
-

8. Success Metrics (KPIs)

- Number of plans created
 - Task completion rate
 - Active contributors per plan
 - Time-to-first-task
 - 30 / 60 / 90-day retention
-

USER JOURNEY

Journey 1: First-Time Planner (Plan Owner)

Step 1: Sign Up / Login

- User signs up or logs in
- Creates or joins a workspace

Step 2: Create a Plan

- Click "Create Plan"
- Choose:
 - Blank plan
 - Template (Software, Event, Collection, Marketing, Operations, etc.)

Step 3: Structure the Plan

- Add phases or modules (optional)
- Define milestones
- Customize statuses and fields

Step 4: Add Tasks

- Create tasks and sub-tasks
- Assign contributors
- Set due dates and dependencies

Step 5: Select View

- Switch between Kanban, List, Calendar, or Gantt

Step 6: Invite Contributors

- Invite users via email or link
- Assign roles (Contributor / Observer)

Step 7: Track & Adjust

- Monitor progress and blockers
 - Receive notifications
 - Adjust timelines or priorities
-

Journey 2: Contributor (Team Member)

1. Receives invitation
 2. Joins plan
 3. Views assigned tasks
 4. Updates task status
 5. Collaborates via comments
 6. Completes tasks
-

Journey 3: Observer (Stakeholder)

1. Granted observer access
 2. Views plan dashboard
 3. Tracks milestones and progress
 4. No editing permissions
-

Journey 4: Ongoing Planning & Execution

- Regular reviews using dashboard or Gantt view
 - Identify risks via blockers and overdue tasks
 - Re-plan dynamically as scope changes
 - Complete and archive plan
-

9. Future Enhancements (Post-MVP)

- AI-assisted planning and task breakdown
- Risk and delay prediction

- Automation rules (If X → Then Y)
 - Time tracking and budget management
 - Integrations (Slack, Email, Calendar, Webhooks)
 - Public or external-facing plan views
-

10. Assumptions & Constraints

- Users may not follow Agile or Scrum
 - Must be intuitive for non-technical users
 - Configuration over enforced methodology
-

11. Open Questions

- Should dependencies be hidden by default?
 - How deep should AI be in MVP?
 - Is white-labeling required for early adopters?
-

GoPlan AI – MVP Scope

This section defines the **minimum viable AI capabilities** required to deliver real value while keeping implementation focused, safe, and enterprise-ready.

1. MVP Objectives

The GoPlan AI MVP aims to: - Reduce planning time for users - Help non-expert users create structured plans - Improve task clarity and consistency - Introduce AI safely with human-in-the-loop

Non-goals (explicitly out of scope for MVP): - Fully autonomous execution - AI making irreversible changes without approval - Complex multi-agent optimization

2. AI Capabilities Included in MVP

2.1 Intent-Based Plan Creation (Core Feature)

User input examples: - "Plan a product launch" - "Organize a fundraising event" - "Build MVP for payment app"

AI actions: - Detect CREATE_PLAN intent - Identify plan type (software, event, ops, collection, etc.) - Generate a **Draft Plan** with: - Suggested phases (optional) - Core tasks - Basic milestones

User control: - User must review and approve before publishing

2.2 Auto Task Breakdown (High-Impact)

Input: High-level goal or task

AI output: - Logical task decomposition - Suggested task titles and descriptions - Suggested ordering (no hard dependencies yet)

Example:

"Prepare audit readiness"

→ AI suggests tasks for documentation, compliance checks, internal review

2.3 Smart Task Suggestions (Context-Aware)

AI observes: - Existing tasks in the plan - Selected template - Past similar plans (if available)

AI suggests: - Missing tasks - Common next steps

This is **suggest-only**, never auto-create without user action.

2.4 Natural Language Task Input

Users can type: - "Add task to book venue by March 10" - "Assign design review to Ali"

AI parses: - Task title - Due date - Assignee

Then creates or updates the task.

2.5 Weekly AI Summary (Read-Only)

AI generates: - Weekly progress summary - Blocked tasks - Upcoming deadlines

Delivered via: - Dashboard widget - Optional email

3. AI Capabilities Explicitly Excluded from MVP

- Auto task reassignment

- Auto dependency creation
- Risk prediction
- Timeline auto-adjustment
- Sentiment analysis

These are Phase 2+ features.

4. AI Interaction Modes (MVP)

Default Mode: Suggestion Mode

- AI suggests actions
- User confirms

Assisted Mode (Optional Toggle)

- AI creates drafts automatically
- User edits before publish

Autonomous mode is **not included** in MVP.

5. AI Guardrails & Safety (MVP)

- Role-based permission checks
 - Draft-first creation for plans
 - Explicit confirmation for writes
 - Full logging via Activity_Log
 - Explain-why for AI suggestions
-

6. MVP Technical Components

Required

- Intent classification model
- Prompt templates for plan & task generation
- Context builder (Plan + Tasks + History)
- GoPlan Core API integration

Optional (Nice-to-have)

- Vector DB for similar plan retrieval
 - Basic prompt versioning
-

7. MVP Success Metrics

- % of plans created using AI
 - Time-to-first-task reduction
 - User acceptance rate of AI suggestions
 - AI-generated task edit rate
-

8. MVP Rollout Strategy

1. Internal / beta users only
 2. Suggestion Mode only
 3. Limited plan types (Software, Event, Ops)
 4. Gradual expansion based on feedback
-

9. MVP Exit Criteria

The MVP is considered successful when: - Users trust AI-generated plans - AI suggestions reduce planning effort - No major permission or safety issues arise

GoPlan AI – Intent Schema & Prompt Contracts

This section defines the **formal interface between GoPlan AI and the GoPlan Core system**. These contracts ensure AI behavior is predictable, auditable, and safe for production use.

1. Design Principles

- AI never writes directly without approval
 - All AI outputs must be machine-readable
 - Explicit intent > hidden inference
 - One intent per request (MVP)
 - Deterministic schema, flexible content
-

2. Intent Schema (Canonical)

Every AI request must resolve into this structure.

```
{  
  "intent_id": "uuid",
```

```

    "intent_type": "CREATE_PLAN | ADD_TASK | UPDATE_TASK | ASK_SUMMARY |
SUGGEST_TASKS",
    "confidence": 0.0,
    "source": "chat | ui | api",
    "actor": {
        "user_id": "string",
        "role": "planner | contributor | observer"
    },
    "context": {
        "workspace_id": "string",
        "plan_id": "string | null",
        "existing_tasks": "optional",
        "history_window": "optional"
    },
    "entities": {},
    "constraints": {},
    "timestamp": "ISO-8601"
}

```

3. Intent Types (MVP)

3.1 CREATE_PLAN

```

{
  "intent_type": "CREATE_PLAN",
  "entities": {
    "plan_name": "string",
    "plan_type": "software | event | ops | collection | generic",
    "goal": "string",
    "timeline": "string | null",
    "team": ["user_id"]
  }
}

```

Expected AI behavior: - Generate plan draft - Suggest phases, tasks, milestones - No auto-publish

3.2 ADD_TASK

```

{
  "intent_type": "ADD_TASK",
  "entities": {
    "task_title": "string",

```

```
    "description": "string | null",
    "due_date": "ISO-8601 | null",
    "assignee": "user_id | null"
  }
}
```

3.3 UPDATE_TASK

```
{
  "intent_type": "UPDATE_TASK",
  "entities": {
    "task_id": "string",
    "fields": {
      "status": "string | null",
      "due_date": "ISO-8601 | null",
      "assignee": "user_id | null"
    }
  }
}
```

3.4 ASK_SUMMARY

```
{
  "intent_type": "ASK_SUMMARY",
  "entities": {
    "scope": "plan | workspace",
    "time_range": "week | month"
  }
}
```

3.5 SUGGEST_TASKS

```
{
  "intent_type": "SUGGEST_TASKS",
  "entities": {
    "based_on": "goal | existing_tasks",
    "limit": 10
  }
}
```

4. Prompt Contract Structure

All prompts must follow a **3-layer structure**.

4.1 System Prompt (Static)

Responsibilities: - Define GoPlan domain - Enforce schema output - Prevent unsafe actions

Key rules: - Output JSON only - One intent only - No explanations outside schema

4.2 Context Prompt (Dynamic)

Injected data: - User role - Workspace rules - Existing plan snapshot - Recent activity (Activity_Log)

4.3 User Prompt (Raw Input)

Natural language input from user.

5. Example Prompt Contract (CREATE_PLAN)

System Prompt

```
You are GoPlan AI. Your task is to convert user input into a valid intent
schema.
Rules:
- Output must be valid JSON
- Use the provided schema
- Never create or update data directly
- If unsure, reduce confidence
```

Context Prompt

```
User Role: Planner
Workspace Type: SME
Existing Plans: 3
```

User Prompt

"Organize a fundraising dinner in April"

Expected Output

```
{
  "intent_type": "CREATE_PLAN",
  "confidence": 0.94,
  "entities": {
    "plan_name": "Fundraising Dinner",
    "plan_type": "event",
    "goal": "Raise funds through a dinner event",
    "timeline": "April"
  }
}
```

6. Post-Intent Execution Contract

Once intent is resolved:

1. Validate role permissions
2. Create DRAFT entities only
3. Store intent + output in Activity_Log
4. Require explicit user approval to apply

7. Error & Fallback Handling

- Low confidence (<0.6) → ask clarification
- Missing entities → partial draft
- Permission mismatch → read-only response

8. Why This Matters

This contract: - Makes AI behavior testable - Prevents hallucinated actions - Enables multi-agent expansion later - Keeps GoPlan enterprise-safe

9. Next Evolution (Post-MVP)

- Multi-intent chaining
 - Intent memory per user
 - Agent-to-agent contracts
 - Autonomous rule execution
-

MCP-Compatible Specification (GoPlan)

1. MCP Overview

GoPlan adopts **Model Context Protocol (MCP)** to standardize how AI agents interact with projects, tasks, and planning logic.

MCP enables: - Tool-based execution (no direct DB writes by AI) - Deterministic, auditable actions - Multi-agent extensibility - Channel-agnostic usage (chat, API, Slack, WhatsApp)

2. MCP Server Role

GoPlan MCP Server acts as: - Single source of truth - Policy enforcer - Tool registry - Execution layer

AI agents: - Interpret intent - Call MCP tools - Never mutate state directly

3. MCP Context Objects

3.1 ProjectContext

```
{
  "project_id": "uuid",
  "name": "string",
  "status": "planned | active | blocked | completed",
  "domain": "software | ops | event | personal | custom",
  "start_date": "date",
  "end_date": "date",
  "tags": ["string"],
  "health_score": 0.0
}
```

3.2 TaskContext

```
{
  "task_id": "uuid",
  "project_id": "uuid",
  "title": "string",
  "status": "todo | in_progress | blocked | done",
  "priority": "low | medium | high | critical",
  "assignee": "user_id",
  "due_date": "date",
  "dependencies": ["task_id"]
}
```

4. MCP Intent Envelope

All AI outputs MUST conform to this envelope:

```
{
  "intent_type": "create_task | create_project | update_task | query | plan | suggest",
  "confidence": 0.0,
  "needs_clarification": false,
  "clarification_question": null,
  "entities": {},
  "proposed_actions": []
}
```

5. MCP Tools (Authoritative)

5.1 Project Tools

- project.create
- project.update
- project.get
- project.list

5.2 Task Tools

- task.create
- task.update
- task.move
- task.get

- `task.list`

5.3 Planning & AI Tools

- `plan.generate_tasks`
- `plan.analyze_risks`
- `plan.suggest_priorities`

6. Tool Invocation Contract

```
{
  "tool": "task.create",
  "arguments": {
    "project_id": "uuid",
    "title": "string",
    "priority": "high",
    "due_date": "date"
  }
}
```

Rules: - Tools validate permissions - Tools enforce schema - Tools return structured results only

7. Multi-Agent Compatibility

GoPlan MCP supports multiple agents:

Agent	Responsibility
Intent Agent	Parse user input
Planner Agent	Decompose goals
Executor Agent	Call MCP tools
Analyst Agent	Insights & risk

Agents communicate ONLY via MCP envelopes.

8. Safety & Determinism

- No free-form execution
- No hidden side effects
- All actions traceable
- AI suggestions require user confirmation (MVP)

9. Example Flow (User → MCP)

1. User: "Prepare product launch next month"
 2. Intent Agent → MCP Intent Envelope
 3. Planner Agent → `plan.generate_tasks`
 4. Executor Agent → staged `task.create` calls
 5. User approves → execution
-

10. Why MCP for GoPlan

- Future-proof agent architecture
- Enterprise-ready
- Works with Cursor, Cline, Claude, OpenAI Agents
- Clean separation: AI thinks, system acts